

CampusHub SOLID 检查清单

团队： 暴风星云裂 | 项目： CampusHub | 日期： 2026年5月16日

阶段： P3 详细设计

检查对象： 01-class-diagram.md 中的第一版核心类图

1. 检查结论

我们当前这版核心类图整体上是比较克制的，主业务对象划分清晰，暂时**没有明显“大面积违反 SOLID”**的情况。

不过，类图仍处在“第一版核心业务骨架”阶段，因此有几处**潜在设计风险**值得在后续详细设计中继续约束，避免落地成代码后演变为真正的问题。

2. SOLID 逐项检查

SOLID 原则	检查问题	是否发现明显违背	说明	修正/约束方案
S - 单一职责原则	有没有类承担了过多职责？	部分存在风险，但当前不算明显违背	Order 是当前模型中的业务中心，既承载订单基本信息，又暴露了确认、完成、取消、争议等多个业务动作。就类图阶段而言这是合理的，因为这些动作都围绕订单生命周期；但如果在实现中继续把通知、信用分变更、举报联动等逻辑也塞进 Order 或 OrderService ，就会变成职责过重。	保持 Order 只表达订单状态与订单生命周期动作；通知写入交给 NotificationService ，信用分变更交给 ReviewService / CreditService ，举报处理交给 ReportService 。
O - 开闭原则	新增需求类型时，是否必须频繁修改旧代码？	存在潜在风险	Task 当前使用统一的 category 字段描述不同需求类型，这对 MVP 是合理的；但如果后续把“快递代取、二手交易、失物招领”等差异化校验规则都直接堆进一个 TaskService 的大分支里，就会导致每新增一种类型都要修改旧逻辑。类图本身尚未体现这种违背，但实现阶段需要警惕。	在后续详细设计中为“不同需求类型校验/组装”预留策略模式或分类处理器接口，避免把所有类别差异写成超长 if-else 。
L - 里氏替换原则	有没有不合理继承，导致子类不能替代父类？	未发现明显违背	我们当前这版核心类图几乎没有使用继承层次，这是比较稳妥的。尤其是没有把“管理员”强行建模成复杂的 AdminUser extends User 体系，而是更适合通过角色字段区分。	维持当前做法，除非后续真的出现稳定、清晰的“is-a”关系，否则不要为了画 UML 而强行引入继承。

SOLID 原则	检查问题	是否发现明显违背	说明	修正/约束方案
I - 接口隔离原则	有没有过胖接口，逼迫调用方依赖不需要的方法？	当前类图阶段未发现明显违背，但服务层未来有风险	当前交付物主要是实体类图，还没有展开大量接口，因此暂时没有明显的“胖接口”问题。但后续如果把订单相关能力统一塞进一个超大的 <code>OrderService</code> 接口，例如同时暴露申请接单、确认接单、聊天、通知、评价联动等全部方法，就会出现接口过重。	后续服务层设计时按业务边界拆分接口，例如 <code>ApplicationService</code> 、 <code>OrderService</code> 、 <code>ChatService</code> 、 <code>NotificationService</code> 分开设计。
D - 依赖倒置原则	高层模块是否直接依赖低层具体实现？	当前类图阶段未发现明显违背	目前类图只描述核心业务对象，尚未把 <code>Controller</code> 、 <code>Service</code> 、 <code>Mapper</code> 和具体基础设施实现画进去，因此没有看到“高层直接依赖数据库实现类”这类明确违背。P2 中已经约束了 <code>Controller</code> 不直接访问数据库，这一点方向正确。	在后续分层设计中继续坚持 <code>Controller -> Service -> Mapper</code> ，业务服务通过接口协作，避免高层业务直接依赖具体存储或消息实现。

3. 真实存在的设计关注点

3.1 `Order` / `OrderService` 最容易变成“上帝对象”

原因：

- 订单处在主业务链中心
- 聊天、通知、评价、争议都和订单有关
- 初学者实现时很容易把所有逻辑都堆到一个服务里

风险表现：

- 一个服务类里既处理状态流转，又处理消息发送，又写通知，又联动信用分
- 方法越来越长，事务边界越来越混乱

建议：

- 订单状态流转由 `OrderService` 负责
- 订单内聊天由 `ChatService` 负责
- 系统通知由 `NotificationService` 负责
- 评价与信用联动由 `ReviewService` / `CreditService` 负责

3.2 `Task` 的类别差异逻辑未来可能破坏开闭原则

原因：

- `Task` 是统一模型
- 但不同类别需求有不同字段校验规则

风险表现：

- 在一个大方法里判断 `if category == 快递代取`
- 然后继续 `else if 二手交易`
- 再继续 `else if 失物招领`

这种写法在首版能跑，但扩展性会越来越差。

建议：

- 保持数据库主表统一没有问题
- 但业务校验层建议拆成“按类别处理”的策略类或处理器类

3.3 Report 当前采用通用举报目标模型，后续实现时要避免职责失控

原因：

- Report 通过 `targetType + targetId` 支持举报任务、消息、评价、用户
- 这是常见且合理的通用建模方式

风险表现：

- 后续如果把所有目标校验、快照提取、处罚联动都塞进一个 `ReportService.handle()` 大方法里，会迅速变复杂

建议：

- 维持当前统一举报模型
- 但处理不同举报目标时，应拆出目标解析或处理策略，避免一个方法处理所有目标类型

4. 本次检查中明确“未强行判定违背”的点

下面这些点在课程作业里容易被机械地写成“违反 SOLID”，但按当前类图的真实情况，不应强行这么写：

- Order 有多个业务动作，不等于已经违反单一职责原则
说明：这些动作都属于订单生命周期，本身仍在同一业务边界内。
- 没有使用继承，不等于“缺少面向对象设计”
说明：对我们的项目来说，克制使用继承反而更合理。
- Task 用统一实体建模不同需求类型，不等于已经违反开闭原则
说明：真正的问题不在实体统一，而在后续是否把差异逻辑写成无法维护的大分支。
- 当前类图没有明确接口抽象，不等于已经违反依赖倒置原则
说明：类图阶段重点是业务骨架；依赖倒置主要在后续服务层设计里落实。

5. 最终结论

这版核心类图作为 P3 的第一版业务骨架是**基本合理的**，没有为了“看起来高级”而强行引入复杂继承，也没有明显出现职责完全混乱的设计。

真正需要警惕的不是“类图已经严重违反 SOLID”，而是：

- 后续实现时把过多逻辑塞进 `OrderService`
- 用超长 `if-else` 处理所有任务类别差异

- 在举报处理里把所有目标类型逻辑写进一个超大方法

因此，本交付物的重点不是“证明 AI 犯了很多错”，而是**诚实地指出当前设计基本可行，同时标出最容易在实现阶段演变为 **SOLID** 问题的真实风险点。**